```
# ============================================================
# AI ASSESSMENT 1
# Logical Reasoning and Knowledge-Based Agents
# Complete Python Program for all 4 Case Studies
# ============================================================


# ============================================================
# CASE STUDY 1 - SMART MEDICAL DIAGNOSIS SYSTEM
# Forward Chaining and Backward Chaining
# ============================================================

def medical_diagnosis():
    print("\n" + "=" * 65)
    print("CASE STUDY 1 - SMART MEDICAL DIAGNOSIS SYSTEM")
    print("=" * 65)

    # Facts about Patient A
    facts = {
        "Fever": True,
        "Cough": True,
        "Rash": False,
        "Breathlessness": True
    }

    # Knowledge base rules
    rules = [
        (["Fever", "Cough"], "Flu"),
        (["Fever", "Rash"], "Measles"),
        (["Cough", "Breathlessness"], "Pneumonia")
    ]
```

```
print("\nInitial Facts:")
for fact, value in facts.items():
    print(f"{fact} = {value}")

# -------------------------
# Forward Chaining
# -------------------------
print("\n--- Forward Chaining ---")

derived = set()

changed = True

while changed:
    changed = False

    for conditions, conclusion in rules:

        if all(facts.get(condition, False) for condition in conditions):

            if conclusion not in derived:
                derived.add(conclusion)
                facts[conclusion] = True
                changed = True

                print(
                    "Rule Fired:",
                    " AND ".join(conditions),
                    "->",
                    conclusion
```

```
            )

print("\nPossible Diagnoses:")

for diagnosis in ["Flu", "Measles", "Pneumonia"]:
    if diagnosis in derived:
        print(diagnosis, "= TRUE")
    else:
        print(diagnosis, "= Not Derivable")

# -------------------------
# Backward Chaining
# -------------------------
print("\n--- Backward Chaining ---")

goal = "Pneumonia"

print("Goal:", goal)

# Find rule that concludes Pneumonia
for conditions, conclusion in rules:

    if conclusion == goal:

        print(
            "Required conditions:",
            " AND ".join(conditions)
        )

        if all(facts.get(condition, False) for condition in conditions):
            print("All conditions are TRUE.")
```

```
            print("Therefore", goal, "is PROVED.")
        else:
            print("Required conditions are not satisfied.")


# ============================================================
# CASE STUDY 2 - AUTONOMOUS TRAFFIC MANAGEMENT AGENT
# Unification and Resolution
# ============================================================

def traffic_management():
    print("\n" + "=" * 65)
    print("CASE STUDY 2 - AUTONOMOUS TRAFFIC MANAGEMENT AGENT")
    print("=" * 65)

    # Facts
    facts = {
        "Vehicle(Ambulance)": True,
        "EmergencyType(Ambulance)": True,
        "Vehicle(CarA)": True,
        "Behind(CarA,Ambulance)": True
    }

    print("\nInitial Facts:")

    for fact in facts:
        print(fact)

    # -------------------------
    # Rule 1
    # Vehicle(x) AND EmergencyType(x) -> ClearPath(x)
```

```
# -------------------------

print("\n--- Unification ---")

print("Rule 1:")
print("Vehicle(x) AND EmergencyType(x) -> ClearPath(x)")

# Unification with Ambulance
substitution = {"x": "Ambulance"}

print("\nMatching Vehicle(x) with Vehicle(Ambulance)")
print("MGU =", substitution)

if (
    facts.get("Vehicle(Ambulance)", False)
    and facts.get("EmergencyType(Ambulance)", False)
):
    facts["ClearPath(Ambulance)"] = True

    print("\nDerived:")
    print("ClearPath(Ambulance)")

# -------------------------
# Rule 2
# ClearPath(x) -> GreenSignal(x)
# ClearPath(x) -> NOT RedSignal(x)
# -------------------------

if facts.get("ClearPath(Ambulance)", False):

    facts["GreenSignal(Ambulance)"] = True
```

```
    facts["RedSignal(Ambulance)"] = False

    print("\nRule 2 Fired:")
    print("GreenSignal(Ambulance)")
    print("NOT RedSignal(Ambulance)")

# -------------------------
# Rule 3
# Vehicle(x) AND Behind(x,y) AND GreenSignal(y)
# -> Proceed(x)
# -------------------------

if (
    facts.get("Vehicle(CarA)", False)
    and facts.get("Behind(CarA,Ambulance)", False)
    and facts.get("GreenSignal(Ambulance)", False)
):
    facts["Proceed(CarA)"] = True

    print("\nRule 3 Fired:")
    print("Proceed(CarA)")

# -------------------------
# Resolution
# -------------------------

print("\n--- Resolution Proof ---")

print("Goal: Proceed(CarA)")
print("Negated Goal: NOT Proceed(CarA)")
```

```
print("\nCNF Clauses:")

print(
   "C1: NOT Vehicle(x) OR "
   "NOT EmergencyType(x) OR ClearPath(x)"
)

print(
   "C2: NOT ClearPath(x) OR GreenSignal(x)"
)

print(
   "C3: NOT ClearPath(x) OR NOT RedSignal(x)"
)

print(
   "C4: NOT Vehicle(x) OR "
   "NOT Behind(x,y) OR "
   "NOT GreenSignal(y) OR Proceed(x)"
)

print("\nResolution steps:")
print("Vehicle(Ambulance)")
print("EmergencyType(Ambulance)")
print("       ↓")
print("ClearPath(Ambulance)")
print("       ↓")
print("GreenSignal(Ambulance)")
print("       ↓")
print("Vehicle(CarA) + Behind(CarA,Ambulance)")
print("       ↓")
```

```
    print("Proceed(CarA)")
    print("       ↓")
    print("Proceed(CarA) AND NOT Proceed(CarA)")
    print("       ↓")
    print("Empty Clause")

    print("\nConclusion:")
    print("Proceed(CarA) is PROVED by Resolution.")



# ============================================================
# CASE STUDY 3 - AGRICULTURAL EXPERT REASONING SYSTEM
# CNF Conversion and Resolution
# ============================================================

def agricultural_system():
    print("\n" + "=" * 65)
    print("CASE STUDY 3 - AGRICULTURAL EXPERT REASONING SYSTEM")
    print("=" * 65)

    # Facts
    facts = {
        "SoilDry": True,
        "CropWheat": True
    }

    print("\nInitial Facts:")
    print("SoilDry = True")
    print("CropWheat = True")

    # -------------------------
```

```
# CNF Conversion
# ------------------------

print("\n--- CNF Conversion ---")

print("\nP1:")
print("SoilDry -> IrrigationNeeded")
print("CNF:")
print("NOT SoilDry OR IrrigationNeeded")

print("\nP2:")
print(
   "IrrigationNeeded AND CropWheat "
   "-> ApplyDripMethod"
)
print("CNF:")
print(
   "NOT IrrigationNeeded OR "
   "NOT CropWheat OR ApplyDripMethod"
)

print("\nP3:")
print(
   "NOT ApplyDripMethod AND CropWheat "
   "-> CropAtRisk"
)
print("CNF:")
print(
   "ApplyDripMethod OR "
   "NOT CropWheat OR CropAtRisk"
)
```

```
# -------------------------
# Forward reasoning
# -------------------------

print("\n--- Resolution / Logical Derivation ---")

# P1
if facts.get("SoilDry", False):

    facts["IrrigationNeeded"] = True

    print("\nFrom:")
    print("SoilDry")
    print("NOT SoilDry OR IrrigationNeeded")

    print("Derived:")
    print("IrrigationNeeded")

# P2
if (
    facts.get("IrrigationNeeded", False)
    and facts.get("CropWheat", False)
):

    facts["ApplyDripMethod"] = True

    print("\nFrom:")
    print("IrrigationNeeded")
    print("CropWheat")
```

```
    print("Derived:")
    print("ApplyDripMethod")

# -------------------------
# Resolution Refutation
# -------------------------

print("\n--- Resolution Refutation ---")

print("Goal: ApplyDripMethod")
print("Negated Goal: NOT ApplyDripMethod")

if facts.get("ApplyDripMethod", False):

    print("\nApplyDripMethod is derived from the KB.")

    print("\nTherefore:")
    print("ApplyDripMethod AND NOT ApplyDripMethod")
    print("                ↓")
    print("            Empty Clause")

    print("\nConclusion:")
    print("ApplyDripMethod is PROVED.")

# -------------------------
# Remove SoilDry
# -------------------------

print("\n--- Removing SoilDry ---")

new_facts = {
```

```
        "CropWheat": True
    }

    if not new_facts.get("SoilDry", False):

        print("SoilDry is removed.")
        print("IrrigationNeeded cannot be derived.")
        print("ApplyDripMethod cannot be derived.")
        print("CropAtRisk cannot be automatically derived.")

        print(
            "\nReason: "
            "Not proving ApplyDripMethod does not mean "
            "NOT ApplyDripMethod."
        )


# ============================================================
# CASE STUDY 4 - WUMPUS WORLD KNOWLEDGE AGENT
# Modus Ponens and Forward Chaining
# ============================================================

def wumpus_world():
    print("\n" + "=" * 65)
    print("CASE STUDY 4 - WUMPUS WORLD KNOWLEDGE AGENT")
    print("=" * 65)

    # Percepts
    percepts = {
        "Stench[1,2]",
        "Breeze[1,1]",
```

```
    "Glitter[2,2]"
}

print("\nInitial Percepts:")

for percept in percepts:
    print(percept)

# -------------------------
# Rule 1 - Stench
# -------------------------

print("\n--- Rule 1: Stench ---")

if "Stench[1,2]" in percepts:

    wumpus_locations = {
        "[1,1]",
        "[1,3]",
        "[2,2]"
    }

    print("Stench[1,2] detected.")
    print("Wumpus is adjacent to [1,2].")

    print("Possible Wumpus locations:")

    for location in sorted(wumpus_locations):
        print(location)

# -------------------------
```

```
# Rule 2 - Breeze
# -------------------------

print("\n--- Rule 2: Breeze ---")

if "Breeze[1,1]" in percepts:

    pit_locations = {
        "[1,2]",
        "[2,1]"
    }

    print("Breeze[1,1] detected.")
    print("Pit is adjacent to [1,1].")

    print("Possible Pit locations:")

    for location in sorted(pit_locations):
        print(location)

# -------------------------
# Rule 3 - Glitter
# -------------------------

print("\n--- Rule 3: Glitter ---")

if "Glitter[2,2]" in percepts:

    gold_location = "[2,2]"

    print("Glitter[2,2] detected.")
```

```
    print("Gold is at:", gold_location)

# -------------------------
# Rule 4 - Safety
# -------------------------

print("\n--- Safety Analysis ---")

print(
    "Rule 4:"
    " NOT Wumpus(x,y) AND NOT Pit(x,y)"
    " -> Safe(x,y)"
)

print(
    "\nThe required negative facts are not available."
)

print(
    "Therefore the agent cannot prove "
    "that [1,1], [1,2], or [2,2] is completely safe."
)

# -------------------------
# Path evaluation
# -------------------------

print("\n--- Path Evaluation ---")

path = [
    "[1,1]",
```

```
        "[1,2]",
        "[2,2]"
    ]

    print("Proposed path:")
    print(" -> ".join(path))

    print("\nAnalysis:")

    print("[1,1]:")
    print("Breeze is present; possible Pit nearby.")

    print("[1,2]:")
    print("Pit is a possible location.")

    print("[2,2]:")
    print("Gold is present, but safety is not proven.")

    print("\nFinal Result:")
    print("The complete path is NOT guaranteed to be safe.")


# ============================================================
# MAIN PROGRAM
# ============================================================

def main():

    print("\n")
    print("*" * 70)
    print("       AI LOGICAL REASONING - ASSESSMENT 1")
```

```
print("*" * 70)

# Run Case Study 1
medical_diagnosis()

# Run Case Study 2
traffic_management()

# Run Case Study 3
agricultural_system()

# Run Case Study 4
wumpus_world()

# -------------------------
# Overall Results
# -------------------------

print("\n" + "=" * 70)
print("OVERALL RESULTS")
print("=" * 70)

print("\nCase Study 1:")
print("Flu = TRUE")
print("Pneumonia = TRUE")
print("Measles = NOT DERIVABLE")

print("\nCase Study 2:")
print("ClearPath(Ambulance) = TRUE")
print("GreenSignal(Ambulance) = TRUE")
print("Proceed(CarA) = TRUE")
```

```
    print("\nCase Study 3:")
    print("IrrigationNeeded = TRUE")
    print("ApplyDripMethod = TRUE")

    print("\nCase Study 4:")
    print("Gold = [2,2]")
    print("Possible Wumpus = [1,1], [1,3], [2,2]")
    print("Possible Pit = [1,2], [2,1]")
    print("Complete path safety = NOT PROVED")

    print("\n" + "*" * 70)
    print("               PROGRAM COMPLETED")
    print("*" * 70)


# Start program
if __name__ == "__main__":
    main()
```